INTUITION_OS_MASTER_SPEC.md

PART 6

ENGINEERING BLUEPRINT

DATABASE SCHEMA · API CONTRACTS · PROMPTS · CLAUDE IMPLEMENTATION PLAN

Version 1.0

PURPOSE OF THIS DOCUMENT

This section exists specifically to enable Claude Code to build the MVP end-to-end.

The previous sections define:

- Product
- Vision
- Philosophy
- User Experience
- AI System

This section defines:

- Database
- APIs
- Prompt Templates
- Folder Structure
- Development Order
- Coding Standards

DATABASE DESIGN

Design Philosophy

The database should store:

Facts

Not conclusions.

Store:

Events

Evidence

Interactions

Sessions

Never store:

AI-generated opinions as truth.

USERS TABLE

```
CREATE TABLE users (
    id UUID PRIMARY KEY,

    email VARCHAR(255) UNIQUE NOT NULL,

    name VARCHAR(255),

    avatar_url TEXT,

    provider VARCHAR(50),

    provider_id VARCHAR(255),

    created_at TIMESTAMP NOT NULL,

    updated_at TIMESTAMP NOT NULL
);
```

Purpose:

Identity.

LEARNING_SESSIONS TABLE

```
CREATE TABLE learning_sessions (
    id UUID PRIMARY KEY,

    user_id UUID REFERENCES users(id),

    problem_title TEXT NOT NULL,

    problem_url TEXT NOT NULL,

    difficulty VARCHAR(50),

    tags JSONB,

    started_at TIMESTAMP NOT NULL,

    ended_at TIMESTAMP,

    duration_seconds INTEGER,

    status VARCHAR(50),

    created_at TIMESTAMP NOT NULL
);
```

Status:

ACTIVE

COMPLETED

ABANDONED

SUBMISSIONS TABLE

```
CREATE TABLE submissions (
    id UUID PRIMARY KEY,

    session_id UUID REFERENCES learning_sessions(id),

    result VARCHAR(20),

    created_at TIMESTAMP NOT NULL
);
```

Result:

WA

TLE

MLE

RE

AC

MENTOR_INTERACTIONS TABLE

```
CREATE TABLE mentor_interactions (
    id UUID PRIMARY KEY,

    session_id UUID REFERENCES learning_sessions(id),

    escalation_level INTEGER,

    observation TEXT,

    question TEXT,

    guidance TEXT,

    user_response TEXT,
```

```
    created_at TIMESTAMP NOT NULL
);
```

REFLECTIONS TABLE

```
CREATE TABLE reflections (
    id UUID PRIMARY KEY,

    session_id UUID REFERENCES learning_sessions(id),

    content JSONB,

    created_at TIMESTAMP NOT NULL
);
```

EVIDENCE TABLE

```
CREATE TABLE evidence (
    id UUID PRIMARY KEY,

    user_id UUID REFERENCES users(id),

    session_id UUID REFERENCES learning_sessions(id),

    evidence_type VARCHAR(100),

    confidence VARCHAR(20),

    details JSONB,

    created_at TIMESTAMP NOT NULL
);
```

Evidence Types

```
SOLVED_WITHOUT_HINT
```

```
SOLVED_WITHOUT_EDITORIAL

RECOVERED_FROM_WA

RECOVERED_FROM_TLE

PATTERN_RECOGNIZED

PATTERN_MISSED

USED_EDITORIAL

USED_HINT

NEEDED_HINT_LEVEL_1

NEEDED_HINT_LEVEL_2

NEEDED_HINT_LEVEL_3

OPTIMIZATION_DISCOVERED

EDGE_CASE_FIXED

MULTIPLE_FAILED_ATTEMPTS
```

FUTURE TABLES

Not MVP.

```
topic_progress

recommendations

weekly_reviews

monthly_reviews

learning_paths
```

API CONTRACTS

Base URL:

```
/api/v1
```

AUTH APIs

Google Login

```
POST /auth/google
```

Request:

```
{
  "token": "google_oauth_token"
}
```

Response:

```
{
  "access_token": "",
  "refresh_token": "",
  "user": {}
}
```

Current User

```
GET /auth/me
```

SESSION APIs

Create Session

```
POST /sessions
```

Request:

```
{
  "problem_title": "",
  "problem_url": "",
  "difficulty": "",
  "tags": []
}
```

Response:

```
{
  "session_id": ""
}
```

Update Session

```
PATCH /sessions/{id}
```

Close Session

```
POST /sessions/{id}/close
```

Triggers:

Reflection Generation

Evidence Extraction

Get Session

```
GET /sessions/{id}
```

Session List

```
GET /sessions
```

SUBMISSION APIs

Record Submission

```
POST /sessions/{id}/submissions
```

Request:

```
{
  "result": "WA"
}
```

MENTOR APIs

Request Help

```
POST /mentor/help
```

Request:

```
{
  "session_id": "",
```

```
  "user_input": "",
  "current_level": 1
}
```

Response:

```
{
  "observation": "",
  "question": "",
  "guidance": "",
  "level": 1
}
```

REFLECTION APIs

Get Reflection

```
GET /reflections/{sessionId}
```

PROFILE APIs

Get Profile

```
GET /profile
```

Get Strengths

```
GET /profile/strengths
```

Get Weaknesses

```
GET /profile/weaknesses
```

CONTEXT ASSEMBLY ENGINE

Most important AI component.

Inputs

Current Problem

Current Session

Current Interaction

Relevant Evidence

User History

Context Construction

```
Current Problem

↓

Current Session

↓

Mentor History

↓

Relevant Evidence

↓
```

```
Learning Profile
```

Context Size Rules

Always prioritize:

Current problem

Current struggle

Current evidence

Do not overload prompt.

PROMPT TEMPLATES

Store all prompts in:

```
packages/prompts
```

Mentor Prompt

System:

```
You are a problem-solving mentor.

Your goal is to build intuition.

Do not immediately reveal solutions.

Always:

1. Observe
2. Diagnose
3. Guide
```

```
Never:

- Dump editorials
- Generate final code
- Skip reasoning

Respond in JSON.
```

Mentor Output Format

```
{
  "observation": "",
  "question": "",
  "guidance": "",
  "level": 1
}
```

Reflection Prompt

System:

```
You are generating a learning reflection.

Focus on:

- What happened
- What was learned
- What evidence exists
- What should happen next

Avoid praise.

Avoid personality judgments.

Use evidence only.
```

Evidence Extraction Prompt

System:

```
Extract objective evidence.

Do not infer personality.

Return only evidence.

Output JSON array.
```

PROFILE GENERATION PROMPT

System:

```
Generate a profile summary.

Use evidence only.

Do not hallucinate strengths.

Do not hallucinate weaknesses.
```

CLAUDE IMPLEMENTATION PLAN

Step 1

Create Monorepo.

Step 2

Setup:

```
apps/backend

apps/dashboard

apps/extension

packages/shared

packages/prompts

packages/types
```

Step 3

Implement Authentication.

Google OAuth.

Step 4

Implement Database.

Alembic migrations.

SQLAlchemy models.

Step 5

Implement Session APIs.

Step 6

Implement Extension.

Detect:

LeetCode Problem Page.

Extract metadata.

Create session.

Step 7

Implement Mentor Service.

Provider abstraction.

Claude.

OpenAI.

Gemini.

Step 8

Implement Reflection Service.

Step 9

Implement Dashboard.

Step 10

Polish MVP.

CODING STANDARDS

Backend

Use:

- Type hints everywhere
- Pydantic schemas
- Repository pattern
- Service layer

Avoid:

Business logic inside routes.

Frontend

Use:

- Server Components where possible
- Feature-based structure
- Shared UI package

Avoid:

Massive components.

AI Layer

Prompts in files.

Not code.

Provider abstraction mandatory.

CLAUDE SUCCESS CRITERIA

MVP is complete when:

User can:

1. Login with Google

2. Install extension

3. Open LeetCode problem

4. Session starts automatically

5. Click Need Help

6. Receive mentor guidance

7. Solve problem

8. Reflection generated

9. View history on dashboard

If these 9 actions work correctly, MVP is considered complete.

FINAL CLAUDE DIRECTIVE

When uncertain:

Choose simplicity.

When choosing between:

More features

vs

Better mentorship

Always choose better mentorship.

The entire product exists to improve independent problem-solving ability.

Everything else is secondary.

END OF PART 6